DSA05 – QUERY PROCESSING FOR DATA SCIENCE

ASSIGNMENT – 2

CO2: Python SQL Libraries and Multi-Database CRUD

MeridianLib: Building and Evaluating a Multi-Database Python Application for a University Library Management System

Student detail	Submission detail
Name: Answar Vijes	Register No.: 192324028
Course: DSA05 – Query Processing for Data Science	Assignment: 2
Nature: Individual	Total: 50 Marks
Submission date: _____	Academic year: 2026

Report note: This report provides the design, SQL patterns, Python implementation approach, test plans and reasoned evaluation requested by the assignment. Database-server execution, screenshots, timings and measured exception text cannot be truthfully claimed without running the scripts against configured SQLite, MySQL and PostgreSQL instances. Such evidence is explicitly marked **TO BE RUN / RECORD LOCALLY**; replace those fields with actual outputs before submission.

Contents and submission scope

This report follows the original assignment structure: Tasks 1–11, followed by the Day-4 Design Challenge. It covers the required analysis, schema, connection patterns, CRUD SQL, validation, transaction safety, portability and recommendation. The companion code ZIP requested by the brief is a separate deliverable; this PDF contains representative, reusable code listings and implementation patterns.

Section	Coverage
Task 1	Entities, business rules, CRUD matrix, data-quality issues, acceptance criteria
Task 2	Layered architecture, driver comparison, DB-API methods
Task 3–4	Secure configuration, connection factory, failure handling and retry policy
Task 5–6	Normalised schema, SQL dialect notes, inserts, bulk loading and transactions
Task 7–9	SELECT, UPDATE, DELETE, referential integrity and audit logging
Task 10–11	Portable repositories, testing, benchmarking and recommendation
Day-4 A–D	Impact analysis, scalability, migration, concurrency, reservations and defence

Implementation assumptions

The logical design uses surrogate integer primary keys internally, with externally supplied member codes retained as unique business identifiers. Email addresses are stored in normalised lowercase form after trimming. Dates use ISO-8601 values. A loan represents one borrowed title; the book row stores aggregate physical-copy counts. All monetary values use fixed-precision DECIMAL/NUMERIC, never floating point. The examples use Python 3.9+, SQLite, MySQL 8+ with InnoDB, and PostgreSQL 14+.

Evidence integrity

Where a requirement asks for measured performance, live server errors, screenshots or test-run results, the report gives the exact experiment and a recording table rather than inventing results. Run the included patterns on the actual environment, record the output, and attach screenshots. A fabricated benchmark or claim that an unexecuted test passed would weaken the submission.

Task 1: Problem Analysis, Data Requirements and Acceptance Criteria (3 Marks)

1.1 Entities and relationships

Entity	Key attributes	Relationship / purpose
category	category_id (PK), name (UNIQUE)	One category has many books (1:M).
book	book_id (PK), isbn (UNIQUE), title, author, category_id (FK), publication_year, copies_total, copies_available	Each book belongs to one category; a title row represents all physical copies.
member	member_id (PK), member_code (UNIQUE), full_name, email (UNIQUE), joined_on, status	One member can have many loans and reservations.
loan	loan_id (PK), member_id (FK), book_id (FK), issued_on, due_on, returned_on, renew_count	Each loan belongs to exactly one member and one book.
fine	fine_id (PK), loan_id (FK), amount, assessed_on, paid	A loan can have zero or more fine records; fine history is retained.
audit_log	log_id (PK), table_name, record_key, action, details, logged_at	Stores an immutable record of approved deletions.
reservation (Day-4)	reservation_id (PK), member_id (FK), book_id (FK), reserved_at, status	Member reserves a title while stock is unavailable; one active reservation per member/title.

Relationship summary: category 1—M book; member 1—M loan; book 1—M loan; loan 1—M fine (or 1—0..1 if policy permits only one consolidated fine); member 1—M reservation; book 1—M reservation. Foreign keys prevent orphan rows.

1.2 Business rules and enforcement

Rule	Enforcement	Reason
BR1 unique ISBN	Database UNIQUE	Prevents duplicate title identity under concurrent inserts.
BR2 unique institutional email	Database UNIQUE + Python format validation	UNIQUE handles races; Python gives friendly validation.
BR3 status in ACTIVE/SUSPENDED/EXPIRED; default ACTIVE	Database CHECK/DEFAULT	Invalid status must be rejected consistently.
BR4 0 ≤ available ≤ total	Database CHECK	Protects stock invariant for every client.
BR5 due date is issue date + 14 days initially	Python application; optional trigger	Date arithmetic varies by dialect; renewal modifies due date.
BR6 loan references existing member and book	Database FOREIGN KEY	Prevents orphan loans.
BR7 maximum three open loans per member	Python transaction + locking/trigger where suitable	Aggregate count cannot be enforced by a simple row CHECK.
BR8 late fine = ■2 per overdue day	Python transaction / stored procedure	Requires comparison with return date and calculation.
BR9 cannot delete member with open loans or unpaid fines	Python transaction + FK RESTRICT	Business-friendly refusal plus database backstop.
BR10 every member deletion is audited	Python transaction + audit table (or trigger)	Audit row and deletion must commit atomically.
BR11 renewal at most twice	Database CHECK (0–2) + conditional UPDATE	Prevents invalid counter and concurrent over-renewal.
Additional: book stock changes atomically with loan	Python transaction and conditional UPDATE	Avoids issuing without decrementing stock.

1.3 CRUD matrix

Entity	Create	Read	Update	Delete
Member	Register a student with validated institutional email.	Find a member by code/email; list ACTIVE members.	Correct email; change status to SUSPENDED/EXPIRED.	Delete only after checking no open loans/unpaid fines; write audit row.

Entity	Create	Read	Update	Delete
Book	Add a title with unique ISBN and valid category.	Search by title/author/category; check copies available.	Correct title metadata or stock count; never allow negative stock.	Delete only if no loan/reservation history; otherwise archive.
Loan	Issue a book if member active, stock positive and open loans < 3.	Show member's current loans, due dates and overdue report.	Return loan; renew due date up to twice.	Delete test/erroneous loan only under an audited administrative process.

1.4 Data-quality and integrity problems in the supplied sample

Sample issue	Consequence if accepted	Treatment
M002 joined_on uses 15/07/2025 instead of ISO date	Ambiguous parsing; date sort/filter may fail.	Parse explicitly and store 2025-07-15.
M002 status is lowercase active	Inconsistent status comparisons.	Trim and uppercase; validate allowed set.
M004 duplicates M001 email	UNIQUE collision or two identities share one account.	Quarantine and resolve correct identity; do not silently merge.
M005 full_name is empty	Unusable member record and poor audit trail.	Reject or request a valid name.
M006 email lacks valid domain format (@meridian)	Notifications and identity verification fail.	Reject with email validation.
Two books have same ISBN	Ambiguous book identity and stock counts split across rows.	Keep one canonical title row; merge verified stock.
Fluent Python has copies_available = -1	Impossible inventory state.	Reject using CHECK; investigate source count.
Loan 3 refers to unknown member M009	Orphan loan; member history and fines cannot be trusted.	Reject until member is verified.
Loan 4 due_on precedes issued_on	False overdue calculation and invalid loan timeline.	Reject; recompute due date.
Loan 5 dates use DD-MM-YYYY	Parsing inconsistency; SQL date operations unreliable.	Convert to ISO 2026-10-31 and 2026-11-14.
Loan 5 references ISBN for a book whose row is valid but loan data date is non-ISO	Date and foreign-key import can fail independently.	Resolve ISBN to book_id and validate dates.

1.5 Acceptance criteria

1. All six core tables and the Day-4 reservation table are created on the target databases with documented keys and constraints. 2. Each database contains at least 6 categories, 60 books, 30 members, 80 loans and 10 fines. 3. At least 10 SELECT, 6 UPDATE, 4 DELETE and 4 INSERT scenarios run using parameterised SQL. 4. At least 20 documented test cases pass on each database. 5. Invalid ISBN, email, status, stock and foreign-key inserts are rejected. 6. Issue and return operations are atomic: injected failure leaves both loan and stock unchanged. 7. Member deletion is blocked when BR9 applies and successful deletions create audit entries. 8. Cross-database query results are equivalent after normalising dates, booleans and ordering. 9. Five benchmark operations are repeated at least three times and mean/spread are reported. 10. No credentials appear in source, logs or screenshots.

1.6 requirements_check.py (sample-data validation)

```
import re
from datetime import date

members = [
    {"code":"M001","name":"Arjun Mehta","email":"arjun.mehta@meridian.edu","joined":"2025-07-15","status":"ACTIVE"},
    {"code":"M004","name":"Divya Nair","email":"arjun.mehta@meridian.edu","joined":"2025-08-03","status":"ACTIVE"},
    {"code":"M005","name":"","email":"neha.k@meridian.edu","joined":"2025-09-10","status":"ACTIVE"},
    {"code":"M006","name":"Imran Khan","email":"imran.k@meridian","joined":"2025-09-12","status":"ACTIVE"},
]
books = [{"isbn":"978-0132350884","available":3,"total":4},
         {"isbn":"978-0132350884","available":2,"total":2},
         {"isbn":"978-1491946008","available":-1,"total":2}]
loans = [{"member":"M009","issued":"2026-10-05","due":"2026-10-19"},
         {"member":"M001","issued":"2026-10-20","due":"2026-10-10"}]
known_members = {m["code"] for m in members}
seen_email, seen_isbn = set(), set()
for m in members:
    email = m["email"].strip().lower()
    if email in seen_email: print("DUPLICATE EMAIL:", m)
    seen_email.add(email)
    if not m["name"].strip(): print("EMPTY NAME:", m)
    if not re.fullmatch(r"[^@\\s]+@meridian\\.edu", email): print("BAD EMAIL:", m)
for b in books:
```

```
    if b["isbn"] in seen_isbn: print("DUPLICATE ISBN:", b)
    seen_isbn.add(b["isbn"])
    if not 0 <= b["available"] <= b["total"]: print("BAD STOCK:", b)
for loan in loans:
    if loan["member"] not in known_members: print("UNKNOWN MEMBER:", loan)
    if date.fromisoformat(loan["due"]) < date.fromisoformat(loan["issued"]):
        print("DUE BEFORE ISSUE:", loan)
```

Expected diagnostic categories: duplicate email, empty name, invalid email, duplicate ISBN, invalid stock, unknown member and due date before issue date. Exact line-by-line output depends on the full sample lists used.

Task 2: Python Database Connectivity, Drivers and DB-API (3 Marks)

2.1 Layered architecture

Layer	Role / path
Python application (MeridianLib)	Business rules, validation, repositories and transaction boundaries.
DB-API 2.0	Common connection/cursor interface: connect, execute, fetch, commit, rollback, close.
Driver	sqlite3 / mysql-connector-python / psycopg2 translates Python calls and values for its database.
Transport	SQLite: direct file access. MySQL and PostgreSQL: client/server network protocol (usually TCP).
Database engine	SQLite embedded engine in a local file; MySQL server; PostgreSQL server.

Architecture flow: Python application → DB-API calls → selected driver → (SQLite file access OR MySQL/PostgreSQL network protocol) → database engine. SQLite is embedded in the application process; MySQL and PostgreSQL run as separate server processes.

2.2 Role of SQL libraries and drivers (approximately 200 words)

A Python SQL library gives an application a controlled way to send SQL statements, bind values, fetch rows and manage transactions. The DB-API 2.0 specification standardises important concepts such as connections, cursors, execute(), fetchone(), commit() and rollback(). This lets application code follow a familiar pattern even when the database product changes. The driver is the database-specific implementation of that interface. It opens the connection, authenticates the user, encodes parameters, sends commands in the database's protocol and converts returned database values into Python objects. SQLite's sqlite3 driver talks directly to an embedded database file. MySQL Connector/Python communicates with a MySQL server, while psycopg communicates with PostgreSQL over its client/server protocol. Changing products is therefore not always a one-line change. Connection arguments and authentication differ; parameter placeholders may be '?' or '%s'; generated-key retrieval and upsert syntax vary; date arithmetic, booleans, exception classes and transaction defaults also differ. A portability layer should isolate these differences in one dialect module. Parameter binding must be used rather than string concatenation, and credentials should be loaded from environment variables or a protected configuration file. The application should depend on repository methods, not on a particular driver.

2.3 Database comparison

Attribute	SQLite	MySQL 8+	PostgreSQL 14+
Python driver	sqlite3 (standard library)	mysql-connector-python or PyMySQL	psycopg2 / psycopg 3
Installation	No server; Python includes driver	Install and run server + driver	Install and run server + driver
Server model	Embedded, file-based	Client/server	Client/server
Default port	Not applicable	3306	5432
Authentication	OS/file permissions; optional application controls	MySQL accounts and authentication plugin	Roles; password, peer, SCRAM-SHA-256 etc.
Connection parameters	database path, timeout	host, port, user, password, database	host, port, user, password, dbname
Placeholders	? (qmark)	%s (format)	%s (format)
Transactions	Explicit transaction; sqlite3 context manager commits/rolls back	Transactions on InnoDB; autocommit configurable	Transactions; autocommit normally off in psycopg
Storage	Single database file; dynamic typing affinity	InnoDB tablespaces and server-managed files	Server-managed relations, WAL and data directory
Concurrency	Many readers; one writer at a time	Concurrent clients; row-level locking in InnoDB	MVCC and row-level locking
Typical use	Local, embedded, small single-site app	Web and general client/server systems	Multi-user systems needing advanced SQL and concurrency

2.4 DB-API methods

Method	Purpose	Driver note
connect()	Creates a database connection.	Arguments and keyword names differ by driver.

Method	Purpose	Driver note
cursor()	Creates an object for executing SQL and fetching rows.	MySQL supports dictionary cursors; psycopg supports RealDictCursor; SQLite supports row_factory.
execute(sql, params)	Runs one parameterised statement.	Placeholder syntax differs; never interpolate untrusted values.
executemany()	Runs a statement for many parameter sets.	May be optimised differently; use one transaction for atomic batches.
fetchone()	Returns next row or None.	Useful for one result; row type depends on cursor configuration.
fetchall()	Returns all remaining rows.	Convenient for small sets; can consume substantial memory.
commit() / rollback()	Persist / undo current transaction.	Autocommit and DDL transaction semantics differ.
close()	Releases cursor or connection resources.	Always close in finally or use a context manager where supported.

env_check.py

```
import importlib
DRIVERS = {"sqlite3": "stdlib", "mysql.connector": "mysql-connector-python",
           "psycopg2": "psycopg2-binary"}
for module, package in DRIVERS.items():
    try:
        m = importlib.import_module(module)
        print(module, "available", getattr(m, "__version__", "version: stdlib"))
        for attr in ("apilevel", "threadsafety", "paramstyle"):
            print(" ", attr, getattr(m, attr, "not exposed"))
    except ImportError:
        print(module, "MISSING - install package:", package)
```

Record the actual version and DB-API attributes from your environment; do not copy guessed version numbers.

Task 3: Establishing Connections and Secure Configuration (4 Marks)

3.1 Configuration policy

Use environment variables such as MLIB_MYSQL_HOST, MLIB_MYSQL_PORT, MLIB_MYSQL_USER, MLIB_MYSQL_PASSWORD and MLIB_MYSQL_DATABASE; define equivalent PG variables. SQLite uses MLIB_SQLITE_PATH. Keep secrets outside version control, restrict configuration-file permissions, and mask credentials in screenshots. Create a dedicated application account for each server with only the privileges required on the MeridianLib database; do not connect as root or postgres.

```
import os
def mysql_config():
    return dict(host=os.getenv("MLIB_MYSQL_HOST", "127.0.0.1"),
                port=int(os.getenv("MLIB_MYSQL_PORT", "3306")),
                user=os.environ["MLIB_MYSQL_USER"],
                password=os.environ["MLIB_MYSQL_PASSWORD"],
                database=os.getenv("MLIB_MYSQL_DATABASE", "meridianlib"),
                connection_timeout=5)
def postgres_config():
    return dict(host=os.getenv("MLIB_PG_HOST", "127.0.0.1"),
                port=int(os.getenv("MLIB_PG_PORT", "5432")),
                user=os.environ["MLIB_PG_USER"],
                password=os.environ["MLIB_PG_PASSWORD"],
                dbname=os.getenv("MLIB_PG_DATABASE", "meridianlib"),
                connect_timeout=5)
def get_connection(db_type):
    if db_type == "sqlite":
        import sqlite3
        c = sqlite3.connect(os.getenv("MLIB_SQLITE_PATH", "meridianlib.db"),
                            timeout=5)
        c.execute("PRAGMA foreign_keys = ON")
        return c
    if db_type == "mysql":
        import mysql.connector
        return mysql.connector.connect(**mysql_config())
    if db_type == "postgres":
        import psycopg2
        return psycopg2.connect(**postgres_config())
    raise ValueError("db_type must be sqlite, mysql or postgres")
```

The three connection scripts should call get_connection(), execute the engine version query (SELECT sqlite_version(), SELECT VERSION(), SELECT version()), print the returned value, and close both cursor and connection in finally. A connection that cannot be reached must produce a clear error without printing the password.

3.2 Meaning of with conn: for sqlite3

For sqlite3, `with conn:` manages a transaction: it commits when the block exits normally and rolls back when an exception escapes. It does not close the connection. Close explicitly in finally or use contextlib.closing around the connection. This distinction matters because an application can finish a transaction yet leave file handles and locks open.

```
from contextlib import closing
import sqlite3
with closing(sqlite3.connect("meridianlib.db")) as conn:
    with conn:
        conn.execute("INSERT INTO category(name) VALUES (?)", ("Databases",))
# closing() closes the connection; inner with controls transaction.
```

3.3 Evidence checklist

Attach: (a) user creation and grants with passwords hidden; (b) masked environment configuration; (c) version output from each database; (d) a short experiment showing a committed insert and a rolled-back insert; and (e) proof that the connection is closed. These must be captured from your own environment.

Task 4: Connection Errors and Exception Handling (3 Marks)

4.1 Failure matrix – scenarios to execute

DB	Induced failure	Expected category	Transient?
SQLite	Database path points to missing parent directory	OperationalError	No; fix path.
SQLite	Open database file read-only and attempt write	OperationalError	No; permissions/configuration.
SQLite	File contains non-SQLite bytes	DatabaseError / file-not-database	No; replace/restore file.
MySQL	Wrong password	Interface/Database driver auth error	No; fix credentials.
MySQL	Wrong host or closed port	OperationalError	Possibly; network may recover.
MySQL	Non-existent database	Programming/Database error	No; create or correct DB.
MySQL	Server stopped	OperationalError	Yes; retry after service returns.
MySQL	User lacks SELECT privilege	Programming/Database error	No; grant privilege.
PostgreSQL	Wrong password	OperationalError / authentication failure	No; fix credentials.
PostgreSQL	Wrong port / server unavailable	OperationalError	Possibly; retry network/service.
PostgreSQL	Non-existent database	OperationalError	No; create or correct DB.
PostgreSQL	Server stopped	OperationalError	Yes; retry after service returns.
PostgreSQL	Insufficient table privilege	InsufficientPrivilege / DatabaseError	No; grant privilege.

Exact exception names, vendor codes and messages depend on driver version and platform. Run each scenario, capture exception class, message and code (errno, pgcode or sqlite_errorcode), and replace this planning matrix with observed values.

4.2 DB-API exception hierarchy

Exception	Meaning / handling
Warning	Non-fatal warning reported by a driver.
Error	Base class for database errors.
InterfaceError	Driver/API misuse or interface failure.
DatabaseError	Database engine reported a problem.
DataError	Invalid data value, range or conversion.
OperationalError	Connection, transaction or operational problem; inspect code before retry.
IntegrityError	Unique, foreign-key, NOT NULL or CHECK violation; do not retry unchanged input.
InternalError	Internal database inconsistency or transaction state problem.
ProgrammingError	SQL syntax, missing table/column or incorrect parameter use.
NotSupportedError	Requested operation is not supported.

4.3 Retry design

```
import logging, time
log = logging.getLogger("meridianlib")
def safe_connect(db_type, retries=3, delay=0.5):
    for attempt in range(retries + 1):
        try:
            return get_connection(db_type)
        except Exception as exc:
            # Production code should classify vendor error codes explicitly.
            transient = is_transient_connection_error(exc, db_type)
            log.warning("connect failed db=%s attempt=%d class=%s",
                        db_type, attempt + 1, type(exc).__name__)
            if not transient or attempt == retries:
                raise
            time.sleep(delay * (2 ** attempt))
```

Only known temporary connection failures (server unavailable, connection reset, selected timeout codes) should be retried. Wrong password, missing database, insufficient privilege, malformed SQL and integrity errors are permanent until

configuration or input changes. Never log connection dictionaries or exception text if it can contain credentials. A retry must not repeat a non-idempotent transaction blindly; reconnect and retry the complete transaction only when its outcome is known to be rolled back or the operation has an idempotency key.

Retry proof: use a mocked connector that fails twice with a classified transient error and succeeds on the third attempt; separately raise an authentication error and assert the call count is one. Record the observed logs and confirm that no password string appears.

Task 5: Schema Design and Table Creation (5 Marks)

5.1 ER model and normalisation

Parent	Cardinality	Child / foreign key
category	1 to many	book.category_id → category.category_id
member	1 to many	loan.member_id → member.member_id
book	1 to many	loan.book_id → book.book_id
loan	1 to many	fine.loan_id → loan.loan_id
member	1 to many	reservation.member_id → member.member_id (Day-4)
book	1 to many	reservation.book_id → book.book_id (Day-4)

The schema is in third normal form under these assumptions: each entity has a single-valued primary key; non-key attributes describe only that entity; category name is stored once in category rather than repeated in every book; member details are not copied into loans; book metadata is stored once per title; fine attributes depend on fine_id; and there are no partial or transitive dependencies among non-key columns. copies_total and copies_available are aggregate inventory attributes deliberately maintained transactionally; they are operational counters, not duplicated member/book descriptive data.

5.2 Portable logical schema (adapt types per dialect)

```
CREATE TABLE category (
  category_id INTEGER PRIMARY KEY, name VARCHAR(60) NOT NULL UNIQUE
);
CREATE TABLE member (
  member_id INTEGER PRIMARY KEY, member_code VARCHAR(20) NOT NULL UNIQUE,
  full_name VARCHAR(120) NOT NULL, email VARCHAR(254) NOT NULL UNIQUE,
  joined_on DATE NOT NULL, status VARCHAR(12) NOT NULL DEFAULT 'ACTIVE'
    CHECK (status IN ('ACTIVE','SUSPENDED','EXPIRED'))
);
CREATE TABLE book (
  book_id INTEGER PRIMARY KEY, isbn VARCHAR(17) NOT NULL UNIQUE,
  title VARCHAR(200) NOT NULL, author VARCHAR(120) NOT NULL,
  category_id INTEGER NOT NULL REFERENCES category(category_id) ON DELETE RESTRICT,
  publication_year INTEGER,
  copies_total INTEGER NOT NULL CHECK (copies_total >= 0),
  copies_available INTEGER NOT NULL CHECK
    (copies_available >= 0 AND copies_available <= copies_total)
);
CREATE TABLE loan (
  loan_id INTEGER PRIMARY KEY,
  member_id INTEGER NOT NULL REFERENCES member(member_id) ON DELETE RESTRICT,
  book_id INTEGER NOT NULL REFERENCES book(book_id) ON DELETE RESTRICT,
  issued_on DATE NOT NULL, due_on DATE NOT NULL, returned_on DATE,
  renew_count INTEGER NOT NULL DEFAULT 0 CHECK (renew_count BETWEEN 0 AND 2),
  CHECK (due_on >= issued_on),
  CHECK (returned_on IS NULL OR returned_on >= issued_on)
);
CREATE TABLE fine (
  fine_id INTEGER PRIMARY KEY,
  loan_id INTEGER NOT NULL REFERENCES loan(loan_id) ON DELETE RESTRICT,
  amount DECIMAL(10,2) NOT NULL CHECK (amount >= 0),
  assessed_on DATE NOT NULL, paid BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE TABLE audit_log (
  log_id INTEGER PRIMARY KEY, table_name VARCHAR(40) NOT NULL,
  record_key VARCHAR(80) NOT NULL, action VARCHAR(20) NOT NULL,
  details TEXT NOT NULL, logged_at TIMESTAMP NOT NULL
);
```

For SQLite, use INTEGER PRIMARY KEY for rowid-backed generated IDs and enable `PRAGMA foreign_keys=ON` on every connection. For MySQL, use `INT AUTO_INCREMENT PRIMARY KEY`, InnoDB, and supported CHECK constraints (enforced in modern MySQL 8). For PostgreSQL, use `GENERATED BY DEFAULT AS IDENTITY` or `BIGSERIAL`. Create tables in parent-before-child order.

5.3 Type mapping

Logical field	SQLite	MySQL	PostgreSQL	Reason
ID	INTEGER PRIMARY KEY	INT AUTO_INCREMENT	INTEGER GENERATED AS IDENTITY	Database-generated integer key.
Email	TEXT / VARCHAR affinity	VARCHAR(254)	VARCHAR(254)	Bounded text with uniqueness.

Logical field	SQLite	MySQL	PostgreSQL	Reason
Money	NUMERIC(10,2)	DECIMAL(10,2)	NUMERIC(10,2)	Exact decimal arithmetic.
Date	TEXT ISO-8601 or DATE affinity	DATE	DATE	Consistent calendar date.
Boolean	INTEGER 0/1	BOOLEAN (TINYINT)	BOOLEAN	Portable true/false meaning.
Long details	TEXT	TEXT	TEXT	Unbounded audit details.
Counter	INTEGER	INT	INTEGER	Integer arithmetic and CHECK.
Timestamp	TEXT ISO-8601	DATETIME/TIMESTAMP	TIMESTAMP	Audit time; store timezone policy consistently.

5.4 Engine notes and dialect differences

SQLite foreign keys are disabled by default in many configurations and must be enabled for every connection with PRAGMA foreign_keys=ON before a transaction. Without it, orphan loans may be inserted and deletes may bypass referential rules. MySQL must use InnoDB (not a non-transactional engine) for foreign keys and transactions. Differences to document: (1) auto-increment syntax; (2) placeholders ? versus %s; (3) date addition syntax; (4) generated-key retrieval; (5) boolean storage; (6) upsert syntax; (7) DDL transaction behaviour; (8) LIKE case sensitivity/collation.

Schema verification should query sqlite_master and PRAGMA table_info in SQLite, and information_schema.columns in MySQL/PostgreSQL. Run create_schema.py twice to prove re-runnability; record the table/column listing from all three systems.

Task 6: Safe Insertion, Bulk Loading and Transactions (4 Marks)

6.1 Seed data and minimum volumes

Generate reproducible synthetic data with random.Random(2026). Create at least 6 categories, 60 books, 30 members, 80 loans and 10 fines per database. Generate parents before children and retain a mapping from business codes/ISBNs to generated primary keys. For reproducibility, use fixed names and dates or a fixed random seed. Use execute() for one demonstration insert and executemany() for bulk rows.

6.2 Placeholder adapter and parameter binding

Driver	Placeholder	Example
sqlite3	?	INSERT INTO member(email) VALUES (?)
mysql-connector	%s	INSERT INTO member(email) VALUES (%s)
psycopg2	%s	INSERT INTO member(email) VALUES (%s)

The repository should use one canonical `%s` placeholder and translate to `?` only for SQLite, or use a query builder that returns the appropriate placeholder. Values must always be passed separately as a parameter sequence. Never use f-strings, `%` formatting or concatenation to insert user values.

```
def adapt_sql(sql, db_type):
    return sql.replace("%s", "?") if db_type == "sqlite" else sql

def insert_member(conn, db_type, code, name, email, joined_on):
    import re
    email = email.strip().lower()
    if not name.strip():
        return "Member name is required"
    if not re.fullmatch(r"[^@\\s]+@meridian\\.edu", email):
        return "Use a valid @meridian.edu email"
    sql = adapt_sql(
        "INSERT INTO member(member_code,full_name,email,joined_on,status) "
        "VALUES (%s,%s,%s,%s,'ACTIVE')", db_type)
    try:
        cur = conn.cursor()
        cur.execute(sql, (code, name.strip(), email, joined_on))
        conn.commit()
        return "Member created"
    except Exception as exc:
        conn.rollback()
        # In production catch the driver's IntegrityError specifically.
        return "Member could not be created: " + type(exc).__name__
```

6.3 String formatting and injection

An f-string containing Sneha O'Brien creates an unescaped quote and can break SQL syntax. Concatenating input such as `' OR '1'='1` can change the meaning of a WHERE clause. Parameter binding treats the entire value as data, so the apostrophe is stored safely and the injection string does not become SQL syntax.

6.4 Invalid insert scenarios

Attempt	Expected protection
Duplicate ISBN	UNIQUE constraint / IntegrityError
Duplicate email	UNIQUE constraint / IntegrityError
NULL required full_name	NOT NULL constraint
Loan with unknown member_id	FOREIGN KEY constraint
Negative copies_available	CHECK constraint
Invalid status value	CHECK constraint
Due date earlier than issue date	CHECK constraint
Invalid date text	Python ISO-date validation; DB type validation where available

Run each attempt on all three databases and record the actual driver exception class and message. Driver error text differs, so a single generic error string should not be assumed.

6.5 Duplicate-ignore syntax

Database	Pattern	Caution
SQLite	INSERT OR IGNORE ...	May suppress more than duplicate-key errors; validate inputs first.
MySQL	INSERT IGNORE ...	Can downgrade some errors to warnings; inspect warnings.
PostgreSQL	INSERT ... ON CONFLICT (isbn) DO NOTHING	Specify the conflict target where possible.

6.6 Transaction demonstration

Insert a batch of 20 loans inside one transaction and deliberately make row 15 invalid. On failure call rollback(); the count of newly inserted rows must be zero. Run the same batch with valid data and commit(); the count must increase by 20. Retrieve generated IDs using cursor.lastrowid for SQLite/MySQL where supported and INSERT ... RETURNING loan_id for PostgreSQL (or use a compatible driver feature).

```
cur = conn.cursor()
try:
    cur.executemany(adapt_sql(
        "INSERT INTO loan(member_id,book_id,issued_on,due_on) "
        "VALUES (%s,%s,%s,%s)", db_type), batch_20)
    conn.commit()
except Exception:
    conn.rollback()
    raise
finally:
    cur.close()
```

Task 7: Selecting and Presenting Records (4 Marks)

7.1 Required SELECT set

No.	Operational question	SQL pattern / key clause
1	Books in a category	JOIN category; WHERE category.name = %s; ORDER BY title
2	Members by status	WHERE status = %s ORDER BY full_name
3	Low stock books	WHERE copies_available < 2
4	Overdue open loans	returned_on IS NULL AND due_on < CURRENT_DATE
5	Members with multiple loans	GROUP BY member_id HAVING COUNT(*) > 1
6	Five most borrowed titles	JOIN loan/book; GROUP BY book; ORDER BY COUNT(*) DESC; LIMIT 5
7	Unpaid fine total per member	JOIN fine/loan; WHERE paid = false; SUM(amount); GROUP BY member
8	Title search	WHERE title LIKE %s; parameter contains search pattern
9	Books never borrowed	LEFT JOIN loan; WHERE loan.loan_id IS NULL
10	User-selected category/title	Parameterized WHERE value; no SQL concatenation

Example: overdue report

```
SELECT l.loan_id, m.member_code, m.full_name, b.title, l.due_on
FROM loan l
JOIN member m ON m.member_id = l.member_id
JOIN book b ON b.book_id = l.book_id
WHERE l.returned_on IS NULL AND l.due_on < CURRENT_DATE
ORDER BY l.due_on, l.loan_id;
```

7.2 Fetch methods and memory

fetchone() is appropriate for a single row such as a member lookup. fetchmany(n) retrieves a controlled batch when processing a large result incrementally. fetchall() is convenient for a small result set but loads all remaining rows into memory. Iterating directly over the cursor streams rows according to driver buffering behaviour and is usually suitable for reports. Use sqlite3.Row, MySQL dictionary=True cursors, or psycopg2.extras.RealDictCursor so fields can be accessed by column name. Print headings, aligned rows and a final row count.

7.3 Cross-database equivalence

Compare sorted, normalised result sets rather than relying on unspecified row order. Normalise dates to ISO strings, booleans to Python bool, decimals to Decimal or fixed strings, and NULL consistently. Potential differences include SQLite's default case-insensitive LIKE for ASCII in common builds versus other engines/collations, NULL sort order, date functions and strict GROUP BY rules. Use explicit ORDER BY and portable expressions; explain any observed mismatch rather than silently discarding it.

Task 8: Updating Records and Verifying Changes (3 Marks)

8.1 Six update operations

Operation	Safe update pattern	Verification
Return a book	UPDATE loan SET returned_on=%s WHERE loan_id=%s AND returned_on IS NULL; then increment stock conditionally in same transaction.	SELECT loan and book before/after; rowcount must be 1 for each.
Renew loan	UPDATE loan SET due_on=due_on + 7 days, renew_count=renew_count+1 WHERE loan_id=%s AND returned_on IS NULL AND renew_count < 2.	Check due date, count and rowcount.
Change email	Validate, lowercase, UPDATE member WHERE member_id=%s.	Select before/after; zero rowcount means not found.
Expire memberships	UPDATE member SET status='EXPIRED' WHERE status='ACTIVE' AND joined_on < cutoff.	Record rowcount and sample affected rows.
Pay fine	UPDATE fine SET paid=TRUE WHERE fine_id=%s AND paid=FALSE.	Check paid value and rowcount.
Correct stock	UPDATE book SET copies_total=%s,copies_available=%s WHERE book_id=%s AND valid values.	CHECK constraints reject invalid range.

Date arithmetic must live in the dialect module: SQLite date(due_on, '+7 days'), MySQL DATE_ADD(due_on, INTERVAL 7 DAY), PostgreSQL due_on + INTERVAL '7 days'.

8.2 Atomic return-book operation

```
def return_book(conn, db_type, loan_id, today):
    cur = conn.cursor()
    try:
        cur.execute(adapt_sql(
            "UPDATE loan SET returned_on=%s "
            "WHERE loan_id=%s AND returned_on IS NULL", db_type),
            (today, loan_id))
        if cur.rowcount != 1:
            raise ValueError("Loan not found or already returned")
        cur.execute(adapt_sql(
            "UPDATE book SET copies_available=copies_available+1 "
            "WHERE book_id=(SELECT book_id FROM loan WHERE loan_id=%s) "
            "AND copies_available < copies_total", db_type), (loan_id,))
        if cur.rowcount != 1:
            raise ValueError("Stock update failed")
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        cur.close()
```

The example demonstrates transaction structure; a production implementation should lock or conditionally update the book row and ensure the stock cap is consistent with the loan row. Inject a failure after the first UPDATE and verify rollback restores returned_on to NULL.

8.3 Rowcount and missing WHERE

Always check rowcount for targeted operations. Updating a row to its existing value can report matched rows or changed rows depending on driver/client flags; MySQL commonly reports changed rows by default, while SQLite/PostgreSQL generally report rows matched/processed. Record the actual driver configuration. Demonstrate a missing-WHERE UPDATE only on a disposable copy inside a transaction, count affected rows, then roll back. Repository methods should require a primary key or explicit predicate and tests should assert the expected rowcount.

Task 9: Deleting Records, Referential Integrity and Safe Deletion (4 Marks)

9.1 Conditional delete scenarios

1. Delete a book that has never been borrowed and has no reservation. 2. Delete returned loans older than a retention date after confirming no dependent fine exists. 3. Delete unpaid-free fine test rows only in a test dataset. 4. Delete obsolete reservations with terminal status and an expiry date. Each DELETE must include a condition, report cursor.rowcount and run in a transaction. Avoid deleting operational history simply to satisfy a row-count target.

9.2 Foreign-key policies

Relationship	Recommended policy	Reason
loan → member	RESTRICT / NO ACTION	Preserves borrowing history and blocks member deletion while loans exist.
loan → book	RESTRICT / NO ACTION	A title with loan history must not disappear.
fine → loan	RESTRICT / NO ACTION	Fine must retain its source loan for dispute/audit.
book → category	RESTRICT	Prevent deletion of a category still used by books.
reservation → member	CASCADE only after explicit safe-delete checks, otherwise RESTRICT	No orphan reservation; policy must align with BR9 and audit.
reservation → book	CASCADE or RESTRICT with explicit cleanup	A removed book must not leave active reservations.

SQLite with foreign keys OFF may allow a parent deletion and leave orphan rows; with foreign keys ON it enforces the declared action. MySQL InnoDB and PostgreSQL enforce foreign keys. RESTRICT/NO ACTION blocks deletion; CASCADE removes child rows; SET NULL preserves the child but clears the reference and requires a nullable FK. Use SET NULL only where the child remains meaningful without its parent.

9.3 Safe member deletion and audit

```
def safe_delete_member(conn, db_type, member_id, now_text):
    cur = conn.cursor()
    try:
        cur.execute(adapt_sql(
            "SELECT COUNT(*) FROM loan WHERE member_id=%s AND returned_on IS NULL",
            db_type), (member_id,))
        if cur.fetchone()[0]:
            raise ValueError("Cannot delete: member has active loans")
        cur.execute(adapt_sql(
            "SELECT COUNT(*) FROM fine f JOIN loan l ON l.loan_id=f.loan_id "
            "WHERE l.member_id=%s AND f.paid=FALSE", db_type), (member_id,))
        if cur.fetchone()[0]:
            raise ValueError("Cannot delete: member has unpaid fines")
        cur.execute(adapt_sql(
            "SELECT member_code,full_name,email,status FROM member WHERE member_id=%s",
            db_type), (member_id,))
        row = cur.fetchone()
        if row is None: raise ValueError("Member not found")
        details = repr(tuple(row))
        cur.execute(adapt_sql(
            "INSERT INTO audit_log(table_name,record_key,action,details,logged_at) "
            "VALUES (%s,%s,%s,%s,%s)", db_type),
            ("member", str(member_id), "DELETE", details, now_text))
        cur.execute(adapt_sql(
            "DELETE FROM member WHERE member_id=%s", db_type), (member_id,))
        if cur.rowcount != 1: raise RuntimeError("Delete did not affect one member")
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        cur.close()
```

This operation must run as one transaction. A refused deletion should leave both member and audit table unchanged. A simulated failure after audit insertion but before deletion must roll back the audit row as well.

9.4 Soft delete versus hard delete (approximately 150 words)

A soft delete changes a member's status to a terminal value such as DELETED or INACTIVE instead of physically removing the row. It is preferable when the library must retain loan history, fine disputes, audit evidence or links to reservations. It also supports accidental-deletion recovery and makes historical reports easier because foreign-key references remain valid. A

soft-deleted member must be excluded from login, borrowing and reservation workflows, and the application must enforce that status consistently. Hard deletion physically removes the row and may be appropriate for duplicate test records or legally required erasure after retention obligations have expired. It reduces active data but can destroy relationships and requires careful cascading or audit capture. For MeridianLib, soft deletion should be the default operational choice; hard deletion should be rare, authorised, checked against BR9, and recorded in audit_log.

Task 10: A Common Multi-Database Application (5 Marks)

10.1 Layered application design

Layer	Responsibility
Configuration	Read environment variables and choose database connection settings.
Connection factory	Open connections and apply engine settings such as SQLite foreign keys.
Dialect module	Own placeholders, date arithmetic, upsert syntax, generated IDs and boolean literals.
Repositories	MemberRepository, BookRepository and LoanRepository expose stable CRUD methods.
CLI / service	Parse --db sqlite|mysql|postgres and invoke repository methods; no raw vendor SQL.

The CLI should expose at least: add member, add book, search books, issue book, return book, renew loan, safe-delete member and overdue-loans report. Changing `--db` should select a connection and dialect; repository method names and business rules remain the same.

10.2 Compatibility table

Issue	SQLite	MySQL	PostgreSQL	Portability handling
Placeholders	?	%s	%s	Dialect adapter.
Generated key	lastrowid	lastrowid	RETURNING id	Dialect method.
Auto increment	INTEGER PRIMARY KEY	AUTO_INCREMENT	IDENTITY	Three schema scripts.
Upsert	INSERT OR IGNORE / ON CONFLICT	INSERT IGNORE / ON DUPLICATE KEY	ON CONFLICT	Dialect method; prefer explicit conflict target.
Date + 7 days	date(x,'+7 days')	DATE_ADD(x, INTERVAL 7 DAY)	x + INTERVAL '7 days'	Dialect expression.
Boolean	0/1 integer	BOOLEAN/TINYINT	BOOLEAN	Convert at repository boundary.
LIKE	Depends on build/collation	Collation dependent	Case-sensitive by default	Define search semantics explicitly.
Autocommit	sqlite3 transaction mode	Connection setting	psycopg default transaction	Set explicitly in connection factory.
Exceptions	sqlite3.Error subclasses	mysql.connector.Error subclasses	psycopg2.Error subclasses	Translate to domain exceptions.
DDL transactions	Transactional behaviour varies by statement/context	Many DDL statements implicitly commit	DDL generally transactional	Test and document.

10.3 Transaction and DDL experiment

For each database: begin a transaction; CREATE TABLE ddl_probe(id INTEGER); roll back; query the catalogue to check whether ddl_probe exists. PostgreSQL supports transactional DDL for ordinary CREATE TABLE. MySQL DDL commonly causes implicit commits. SQLite supports transactional DDL in normal transactional contexts, but test the exact driver and statement mode used. Record actual results, because connection autocommit settings and statement choices affect the observation.

10.4 Separation check

Search the repository/application source for database-type branches. The target is zero branches outside connection_factory.py and dialect.py. All SQL fragments that vary by engine should be returned by dialect methods. Keep validation and business logic in repositories/services.

Task 11: Testing, Performance and Recommendation (4 Marks)

11.1 Test plan – 20 cases

ID	Test / operation	Expected result
T01	Insert valid member	One member created.
T02	Insert duplicate email	Rejected by UNIQUE.
T03	Insert malformed email	Rejected by Python validation.
T04	Insert valid book	One book created.
T05	Insert duplicate ISBN	Rejected by UNIQUE.
T06	Insert negative stock	Rejected by CHECK.
T07	Insert loan for missing member	Rejected by FK.
T08	Insert loan for missing book	Rejected by FK.
T09	Issue with zero stock	No loan; stock unchanged.
T10	Issue to suspended member	Rejected.
T11	Issue fourth active loan	Rejected by BR7.
T12	Return active loan	Loan returned and stock +1.
T13	Return same loan twice	Second return rejected.
T14	Renew loan once	Due date +7; count +1.
T15	Renew loan when count=2	Rejected.
T16	Assess overdue fine	Amount = ■2 × overdue days.
T17	Pay fine	paid becomes TRUE.
T18	Delete member with active loan	Refused; member remains.
T19	Delete eligible member	Member deleted/soft-deleted and audit entry committed.
T20	Injected failure in return transaction	Both loan and stock updates roll back.
T21	Wrong password connection	Permanent failure; no retry.
T22	Temporary server outage	Retry policy attempts within configured limit.
T23	Invalid 20-row batch at row 15	All 20 inserts rolled back.
T24	Valid 20-row batch	All 20 rows committed.

Run every applicable case on all three databases. Test matrix columns required for final submission: ID, CRUD operation, database, input, expected result, actual result, PASS/FAIL. The table above defines expected outcomes; fill actual outcomes from test_crud.py.

11.2 Performance experiment and reporting template

Operation	SQLite mean (ms)	MySQL mean (ms)	PostgreSQL mean (ms)
1,000 inserts; commit each row	TO MEASURE	TO MEASURE	TO MEASURE
1,000 inserts; one executemany + commit	TO MEASURE	TO MEASURE	TO MEASURE
Join/group query on 5,000+ loans, before index	TO MEASURE	TO MEASURE	TO MEASURE
Update 500 rows	TO MEASURE	TO MEASURE	TO MEASURE
Delete 500 rows	TO MEASURE	TO MEASURE	TO MEASURE
Join/group query after index	TO MEASURE	TO MEASURE	TO MEASURE

Use time.perf_counter(), run each operation at least three times, and report mean plus spread (minimum–maximum or standard deviation). Reset to the same starting dataset for each repeat. A likely useful index is `CREATE INDEX idx_loan_book_issued ON loan(book_id, issued_on)` or an index aligned with the actual join/filter; use EXPLAIN to verify that the query planner uses it. Do not claim an index improved speed until measured.

11.3 Interpretation and limitations

A commit per row is expected to be slower than one bulk transaction because each commit incurs durability and round-trip overhead. SQLite may be very fast on a local machine for a small single-user workload because it avoids network latency, but it serialises writers. MySQL and PostgreSQL incur client/server overhead but support multiple concurrent clients and stronger server administration. Indexes speed selected reads at the cost of storage and slower writes. Results depend on hardware, cache warmth, connection placement, data distribution, isolation level and server configuration. A single-machine pilot with small synthetic data is not a production capacity test.

11.4 Usability comparison and recommendation

Criterion	SQLite	MySQL	PostgreSQL
Setup	Easiest; file only	Moderate; server/user setup	Moderate; server/role setup
Administration	Minimal	Mature GUI and server tools	Strong tools and catalogues
Concurrency	Single-writer limitation	Good multi-client support	MVCC; strong concurrent workloads
Scalability	Embedded/single-node	Scales well for common client/server use	Advanced features and large shared workloads
Features	Lightweight SQL subset/extensions	Broad ecosystem	Advanced SQL, constraints, indexing and extensions
Security	File/OS permissions	Accounts, roles and plugins	Roles, authentication and granular privileges
Cost	Open source	Community edition open source	Open source
Portability	Very portable file; concurrency limits	Server deployment required	Server deployment required

Recommendation (single branch, about 500 members): SQLite is a reasonable pilot choice when the library is operated from one desk or has modest write concurrency, provided backups, file permissions and foreign-key enforcement are configured. If several desks must issue simultaneously, choose a server database even at this scale.

Recommendation (three campuses, 20+ librarians): PostgreSQL is the preferred target. It provides a central client/server architecture, MVCC, row-level locking, strong constraints and transactional DDL, making it suitable for concurrent issue/return operations and data integrity. MySQL is a credible alternative where institutional skills, existing hosting or operational tooling favour it. I would reconsider PostgreSQL if the university already standardised on MySQL and demonstrated equivalent reliability, or if a managed platform, staff expertise and cost model strongly favoured another engine.

Day-4 Design Challenge (8 Marks)

A. Impact Analysis (2 Marks)

Component	Impact	Decision	Reason
Connection configuration	Major	MODIFY	Move from local file/server alternatives to central PostgreSQL settings, TLS and pooling.
get_connection()	Major	MODIFY	PostgreSQL only; add pool and connection health checks.
safe_connect()	Major	MODIFY	Network interruption and transaction outcome require reconnect policy and idempotency.
Schema scripts	Breaking	REDESIGN	Add reservation, canonicalised email, uniqueness and large-data indexes.
Repository queries	Major	MODIFY	Locking, conditional stock decrement and reservation queue handling.
Dialect module	Minor	RETAIN/MODIFY	PostgreSQL dialect remains; remove unused SQLite/MySQL paths.
Test suite	Major	MODIFY	Add scale, duplicate merge, concurrency and reservation tests.
Error handling	Major	MODIFY	Distinguish connection loss before commit from uncertain outcome during commit.

Prediction versus observation: before running, predict that SQLite-only assumptions, single-writer behaviour, low-volume tests and non-locking issue logic will not satisfy the new requirement. Run the unchanged application against PostgreSQL and record startup, migration, query and test failures. This report does not claim an observation without an actual run; insert the captured console output and compare it with the prediction.

B. Evaluation of Original Design (2 Marks)

At 20,000 members and 100,000 loans, measure three key queries before and after indexes (for example, member loan history, overdue loans and top borrowed books). Record connection count by instrumenting connection_factory. Run two workers synchronised with a barrier to issue the last copy of one book. The original read-then-write design may oversell if both workers read stock=1 before either decrements it. Simulate a dropped connection after loan insertion but before stock update; verify whether the transaction rolls back or the outcome is uncertain.

C. Adaptation or Redesign (2 Marks)

Migration: read SQLite rows in dependency order, normalise values, insert into PostgreSQL in transactions, and compare per-table row counts plus key sets or checksums. Quarantine duplicate emails after trim/lowercase normalisation and orphan loans for manual resolution; never merge people solely because names look similar. Map status spellings Active/ACTIVE/A to ACTIVE only under a documented mapping rule; unknown codes go to a quarantine table.

Concurrency fix: use a single conditional update as the stock reservation gate. Only one worker can change copies_available from a positive value to one less; the other sees rowcount=0 and must not create a loan. Execute the loan insert and stock decrement in the same transaction.

```
-- PostgreSQL transaction pattern
BEGIN;
UPDATE book
   SET copies_available = copies_available - 1
 WHERE book_id = %s AND copies_available > 0;
-- Application checks rowcount = 1; otherwise ROLLBACK and report unavailable.
INSERT INTO loan(member_id, book_id, issued_on, due_on)
VALUES (%s, %s, CURRENT_DATE, CURRENT_DATE + INTERVAL '14 days');
COMMIT;
```

This conditional UPDATE serialises competing writes to the target row and rechecks the predicate after lock acquisition. For more complex workflows, SELECT ... FOR UPDATE can lock the book row before checking and changing stock. Use a transaction and bounded retry for serialization/deadlock errors.

Connection failure handling: use a PostgreSQL connection pool, health checks and exponential backoff for transient connection failures. If the connection drops during COMMIT, the client may not know whether the transaction committed. Give issue requests an idempotency key stored under a UNIQUE constraint; after reconnect, query that key before attempting the operation again.

Reservation table and rules

```
CREATE TABLE reservation (
  reservation_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
```

```
  member_id INTEGER NOT NULL REFERENCES member(member_id) ON DELETE CASCADE,
  book_id INTEGER NOT NULL REFERENCES book(book_id) ON DELETE CASCADE,
  reserved_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  status VARCHAR(10) NOT NULL DEFAULT 'WAITING'
    CHECK (status IN ('WAITING','READY','CANCELLED','FULFILLED')),
  fulfilled_at TIMESTAMP
);
CREATE UNIQUE INDEX uq_active_reservation
  ON reservation(member_id, book_id)
  WHERE status IN ('WAITING','READY');
CREATE INDEX ix_reservation_queue
  ON reservation(book_id, status, reserved_at);
```

RR1: create a reservation only when copies_available=0, checked in a transaction. RR2: the partial unique index allows at most one active reservation per member/book. RR3: on return, select the earliest WAITING reservation ordered by reserved_at and reservation_id, lock it, then mark READY; do not increase availability if the copy is allocated directly to the waiting member unless the circulation policy explicitly does so. RR4: safe deletion must check active loans/unpaid fines; reservations are either explicitly cancelled and audited or removed by CASCADE only after that check. Book deletion should be blocked while active reservations exist or cancel them transactionally.

D. Evidence-Based Defence (2 Marks)

Component	Decision	Reason	Evidence to attach	Residual risk
Configuration	MODIFY	Central PostgreSQL, TLS, pool	Connection settings and pool metrics	Network outage
Connection factory	MODIFY	Pool and health checks	Connection-count log	Pool exhaustion
safe_connect	MODIFY	Retry transient errors only	Injected outage test	Unknown commit outcome
Schema	REDESIGN	Reservation, canonical email, indexes	Migration counts/key checksums	Bad source identity data
Repositories	MODIFY	Atomic conditional stock decrement	Two-worker race test	Deadlocks under load
Dialect	RETAIN/MODIFY	Keep PostgreSQL SQL only	Zero non-dialect branches	Future vendor change
Tests	MODIFY	Scale and reservation coverage	Updated suite output	Unmodelled edge cases
Error handling	MODIFY	Idempotency and reconnect	Drop-connection test	Operational complexity

Defence (approximately 400 words)

The Day-4 requirement changes MeridianLib from a small pilot into a shared, multi-campus service. The original logical model remains useful, but its deployment assumptions and parts of its transaction design must change. I would retain the core separation between configuration, connection management, repositories and business rules because this structure limits the effect of changing infrastructure. I would also retain parameterised SQL, primary and foreign keys, checks and transaction boundaries. These are not pilot-only practices; they become more important as the number of records and simultaneous users increases.

The connection configuration and factory must be modified to target one central PostgreSQL service. A connection pool avoids opening a new TCP connection for every request and gives the application a controlled maximum number of sessions. Pool size should be based on measured workload and PostgreSQL capacity rather than simply matching the number of desks. TLS, least-privilege roles, statement timeouts and monitoring should be enabled for a networked deployment. The application should not automatically retry every failure: authentication and constraint failures are permanent, while selected network errors may be transient. A lost connection during COMMIT is especially difficult because the client cannot know whether the server committed. An idempotency key makes a repeated issue request safe to reconcile.

The schema requires redesign in two areas. First, reservation is a new entity with foreign keys to member and book, a controlled status, a queue index and a partial unique index for one active reservation per member/title. Second, branch data must be cleaned before it is loaded. Email normalisation should trim whitespace and lowercase values, but collisions must be reviewed rather than blindly merged. Status spellings can be mapped using an approved mapping table. Orphan loans should be quarantined until their member identity is resolved. Migration evidence should include row counts and key-set or checksum comparisons.

The largest correctness risk is concurrent issuing of the last copy. A read-then-write sequence can allow two desks to observe one available copy. A conditional UPDATE that decrements stock only when copies_available is greater than zero is safer: only the transaction that updates one row may insert the loan. Both operations commit or roll back together. A row lock is another valid approach. A two-worker barrier test should demonstrate that exactly one issue succeeds and stock never becomes

negative.

A credible alternative is an ORM such as SQLAlchemy, which can reduce repetitive mapping and provide connection pooling and transaction abstractions. It may be better if the development team needs rapid schema evolution, multiple supported database engines or standardised object mapping. However, an ORM does not remove the need to understand locking, isolation, constraints, query plans or uncertain commit outcomes. For this system, explicit PostgreSQL repositories remain defensible because the concurrency and reservation rules are central and benefit from carefully reviewed SQL. The residual risks are operational: pool sizing, backup/restore, failover, monitoring, data-cleaning decisions and realistic load testing. The final decision should be revisited after the 20,000-member/100,000-loan tests and a production-like concurrency test, not on the basis of a small laptop benchmark alone.

Appendix A: Practical run checklist

Step	Action	Evidence to retain
1	Install Python drivers and verify env_check.py.	Driver versions and DB-API attributes.
2	Create least-privilege MySQL/PostgreSQL users and databases.	Masked SQL / screenshots.
3	Run connection/version scripts.	Three version outputs.
4	Run schema scripts twice and inspect catalogues.	Table and column listings.
5	Seed required minimum records with fixed seed.	COUNT(*) output per table/database.
6	Run insert invalid/duplicate and transaction tests.	Exception matrix, rollback/commit output.
7	Run ten SELECT queries and compare outputs.	Query result tables and comparison report.
8	Run six UPDATE and four DELETE cases.	Before/after rows and rowcounts.
9	Run safe-delete and audit tests.	Refusal, success and rollback evidence.
10	Run 20+ tests on each database.	PASS/FAIL output.
11	Run benchmarks at least three times.	Raw timings, means, spread and chart.
12	Run Day-4 migration, concurrency and reservation tests.	Counts/checksums, race-test output, reservation tests.

Appendix B: Final pre-submission checks

• Replace every TO MEASURE / TO BE RUN item with actual results. • Include the full source code files and SQL scripts in the separate ZIP. • Include screenshots with secrets hidden and short captions explaining each result. • Confirm all programs run from a fresh environment using environment variables. • Cite external documentation consulted. • Use the required filenames: QPDS_A2_192324028_Report.pdf and QPDS_A2_192324028_Code.zip.

Appendix C: Important technical note

This report is a worked design and implementation guide, not proof that database servers were installed or experiments were executed. In particular, actual driver exception names/messages, benchmark timings, query outputs, row counts, screenshots and pass/fail results must come from running the programs. Do not submit the planning tables as measured evidence.

End of Assignment 2 Report